

PROGRAMAÇÃO ORIENTADA A OBJETOS I • C#

Variáveis Primitivas e Referências

Como valores, referências, objetos e arrays se comportam na memória e como isso afeta as atribuições e alterações em C#.

1. Como pensar sobre variáveis

Uma variável é um nome associado a um dado utilizado pelo programa. Em C#, uma distinção fundamental é entre **tipos de valor** e **tipos de referência**.

Essa diferença fica especialmente importante quando uma variável é atribuída a outra. Em tipos de valor, o valor é copiado. Em tipos de referência, a referência para o objeto é copiada.

Stack (pilha)

Região associada principalmente a chamadas de métodos, variáveis locais, parâmetros e informações necessárias à execução dessas chamadas.

LIFO: o último elemento inserido é o primeiro a ser removido.

Heap

Região utilizada para alocação dinâmica de objetos durante a execução do programa. Objetos referenciados podem permanecer alocados enquanto ainda forem necessários.

Em C#, o **Garbage Collector** gerencia a liberação da memória dos objetos que não são mais alcançáveis.

Ideia central: o mais importante não é decorar "stack = valor" e "heap = referência". O ponto fundamental é entender o que acontece com a variável quando ocorre uma atribuição: **o valor é copiado ou a referência é compartilhada?**

2. Tipos de valor

Nos tipos de valor, a atribuição de uma variável para outra produz uma **cópia independente do valor**.

Exemplos

int

float

double

decimal

bool

char

struct

enum

```
int a = 10;
int b = a;

b = 20;

Console.WriteLine(a);
Console.WriteLine(b);
```

10 20

Quando `b = a` é executado, o valor `10` é copiado para `b`. Alterar `b` depois disso não altera `a`.

3. Tipos de referência

Em um tipo de referência, a variável contém uma referência para um objeto. Quando uma variável de referência é atribuída a outra, a referência é copiada; as duas variáveis podem então referenciar o mesmo objeto.

Exemplos

class

array

string

```
class Pessoa
{
    public string Nome;
}

Pessoa p1 = new Pessoa();
p1.Nome = "Alice";

Pessoa p2 = p1;

p2.Nome = "Bob";

Console.WriteLine(p1.Nome);
Console.WriteLine(p2.Nome);
```

Bob Bob

A instrução `Pessoa p2 = p1;` não cria uma nova pessoa. Ela faz `p2` receber a mesma referência armazenada em `p1`. Assim, as duas variáveis acessam o mesmo objeto.

Visualização conceitual



4. Tipos de valor × tipos de referência

Característica	Tipo de valor	Tipo de referência
Exemplo	<code>int</code> , <code>double</code> , <code>bool</code>	<code>class</code> , arrays, <code>string</code>
Ao atribuir uma variável a outra	O valor é copiado	A referência é copiada
Alteração posterior	Não afeta a variável original	Pode afetar o mesmo objeto referenciado
Objeto compartilhado	Não é o comportamento padrão da atribuição	É possível que duas variáveis referenciem o mesmo objeto

5. Conversão de tipos

C# permite converter valores entre tipos compatíveis. A conversão pode ser implícita, explícita ou realizada por métodos de conversão.

5.1 Conversão implícita

É realizada automaticamente quando a conversão é considerada segura para o intervalo de valores envolvido.

```
int x = 10;
double y = x;
```

Nesse caso, um `int` pode ser convertido para `double` sem a necessidade de escrever um cast.

5.2 Conversão explícita — casting

Quando existe possibilidade de perda de informação, a conversão pode exigir uma indicação explícita do programador.

```
double a = 9.7;
int b = (int)a;

Console.WriteLine(b);
```

9

O cast para `int` elimina a parte decimal do valor. Portanto, a conversão não preserva integralmente o valor original.

5.3 Conversão por métodos

A biblioteca do C# fornece métodos específicos para conversão e interpretação de valores.

```
string numero = "123";

int a = int.Parse(numero);
int b = Convert.ToInt32(numero);

int resultado;
bool sucesso = int.TryParse(numero, out resultado);
```

Método	Uso principal
<code>int.Parse()</code>	Converte uma string que deve representar um inteiro válido.
<code>Convert.ToInt32()</code>	Realiza conversões para <code>int</code> usando as regras da classe <code>Convert</code> .
<code>int.TryParse()</code>	Tenta converter sem lançar exceção quando a string não representa um inteiro válido.

6. Arrays são tipos de referência

Arrays em C# são tipos de referência. Por isso, atribuir um array a outra variável copia a referência para o mesmo array.

Declaração e inicialização

```
int[] numeros = new int[3] { 1, 2, 3 };
```

Compartilhamento da referência

```
int[] a = { 1, 2, 3 };  
int[] b = a;  
  
b[0] = 100;  
  
Console.WriteLine(a[0]);  
Console.WriteLine(b[0]);
```

```
100 100
```

a e **b** referenciam o mesmo array. Portanto, alterar uma posição por meio de **b** também altera o array que pode ser acessado por **a**.

7. Resumo

Tipos de valor: a atribuição copia o valor.

Tipos de referência: a atribuição copia a referência para o objeto.

Arrays: são tipos de referência e podem ser compartilhados por várias variáveis.

Conversão implícita: realizada automaticamente quando permitida pelo C#.

Casting: conversão explícita, podendo causar perda de informação.

Parse/TryParse: utilizados principalmente para interpretar strings como valores numéricos.